

Week-10

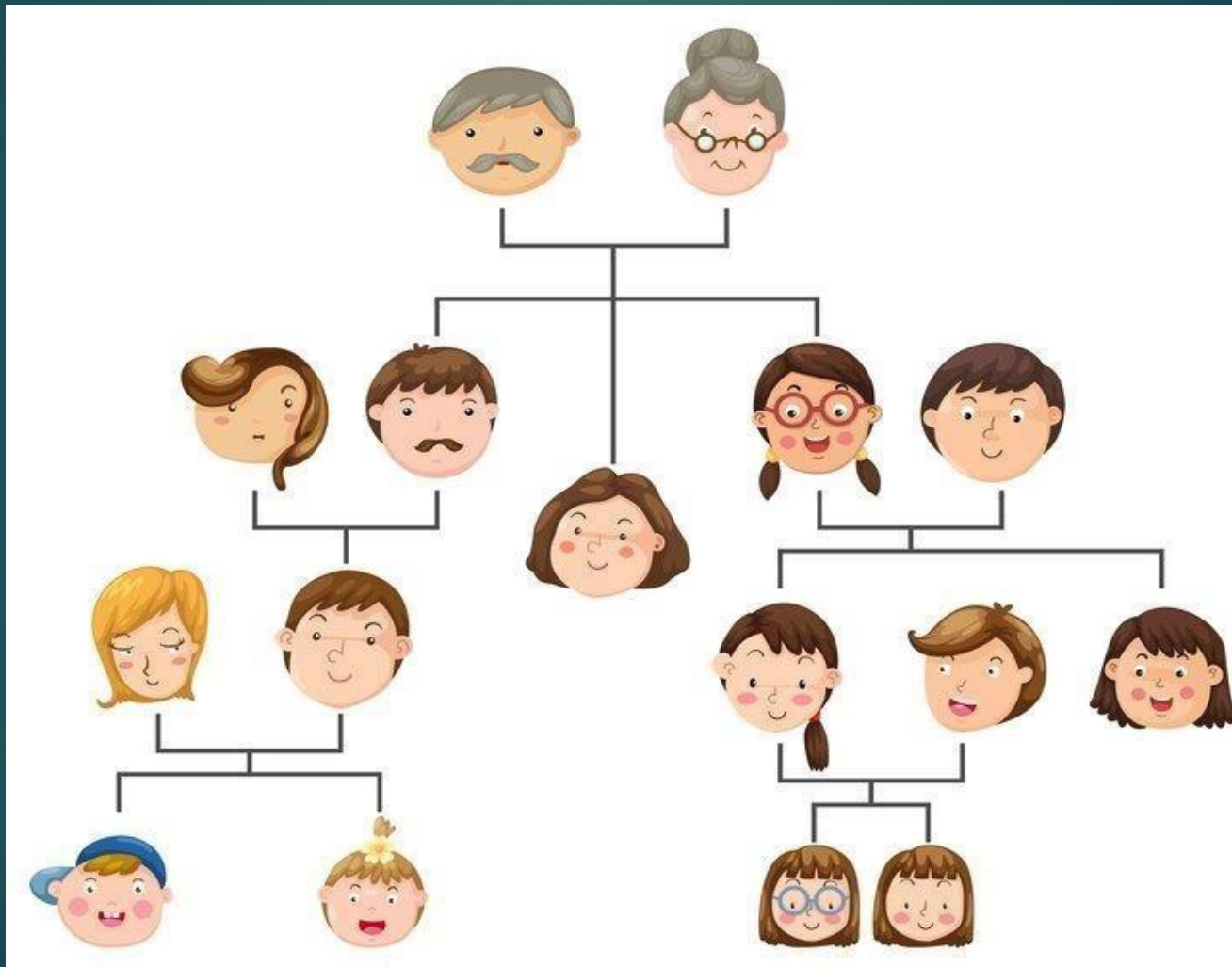
Introduction to Trees,
Binary Tree, Complete
Binary tree

Outline

In this topic, we will cover:

- Definition of a tree data structure and its components
- Concepts of:
 - Root, internal, and leaf nodes
 - Parents, children, and siblings
 - Paths, path length, height, and depth
 - Ancestors and descendants
 - Ordered and unordered trees
 - Subtrees
- Examples

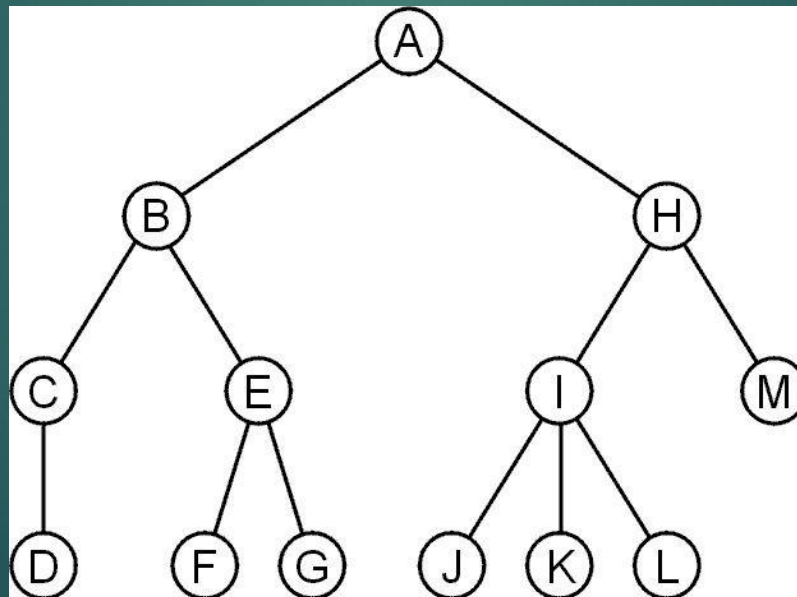
Why Trees Structure?



Trees

A rooted tree data structure stores information in *nodes*

- Similar to linked lists:
 - There is a first node, or *root*
 - Each node has variable number of references to successors
 - Each node, other than the root, has exactly one node pointing to it



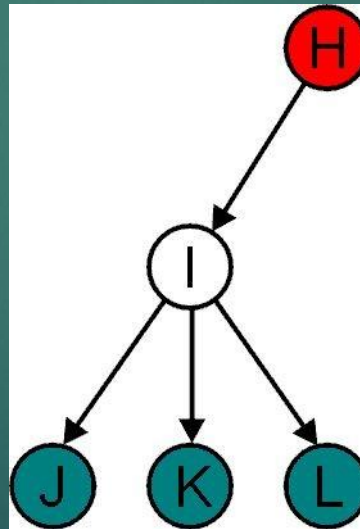
Terminology

All nodes will have zero or more child nodes or *children*

- I has three children: J, K and L

For all nodes other than the root node, there is one parent node

- H is the parent I



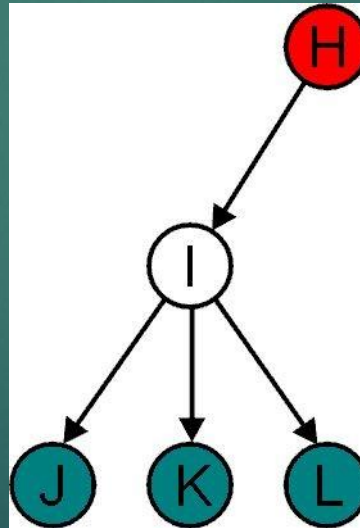
Terminology

The *degree* of a node is defined as the number of its children:

$$\text{deg}(I) = 3$$

Nodes with the same parent are *siblings*

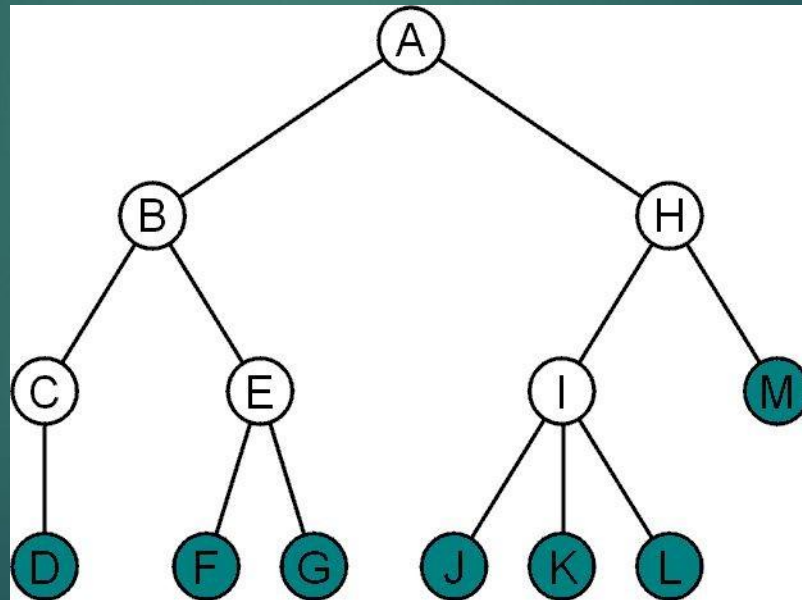
- J, K, and L are siblings



Terminology

Nodes with degree zero are also called *leaf nodes*

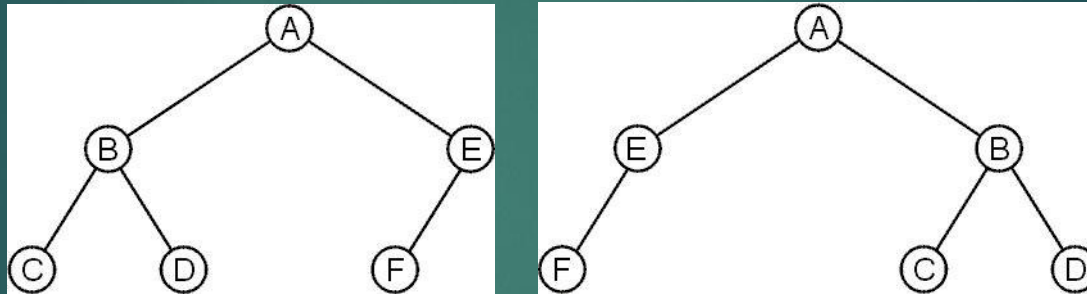
All other nodes are said to be *internal nodes*, that is, they are internal to the tree



Terminology

These trees are equal if the order of the children is ignored

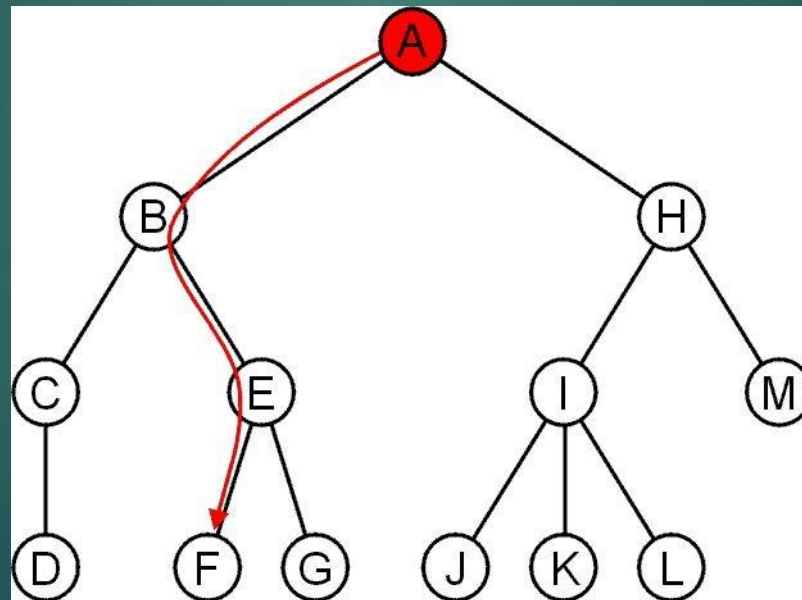
- *unordered trees*



They are different if order is relevant (*ordered trees*)

- We will usually examine ordered trees (linear orders)
- In a hierarchical ordering, order is not relevant

The shape of a rooted tree gives a natural flow from the *root node*, or just *root*



Terminology

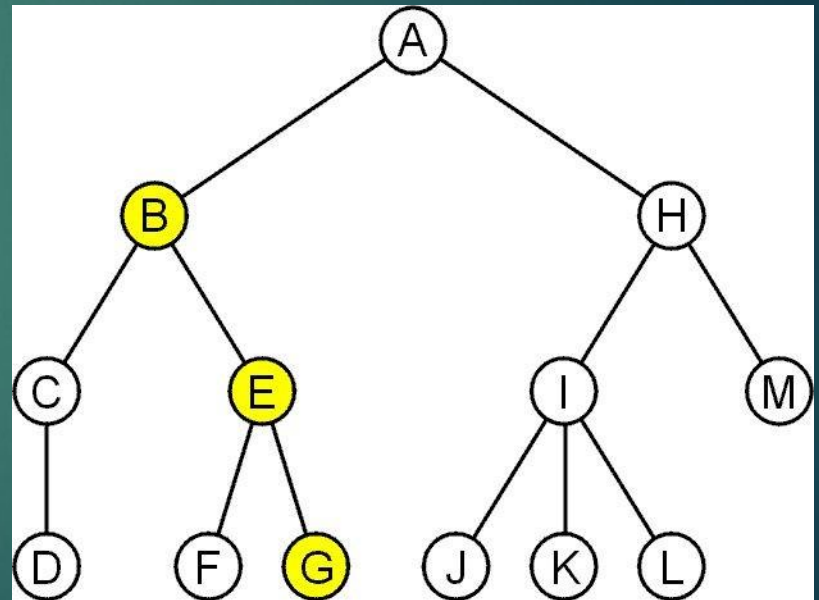
A path is a sequence of nodes

$(a_0, a_1, \dots, a_n)$

where a_{k+1} is a child of a_k is

The length of this path is n

E.g., the path (B, E, G)
has length 2

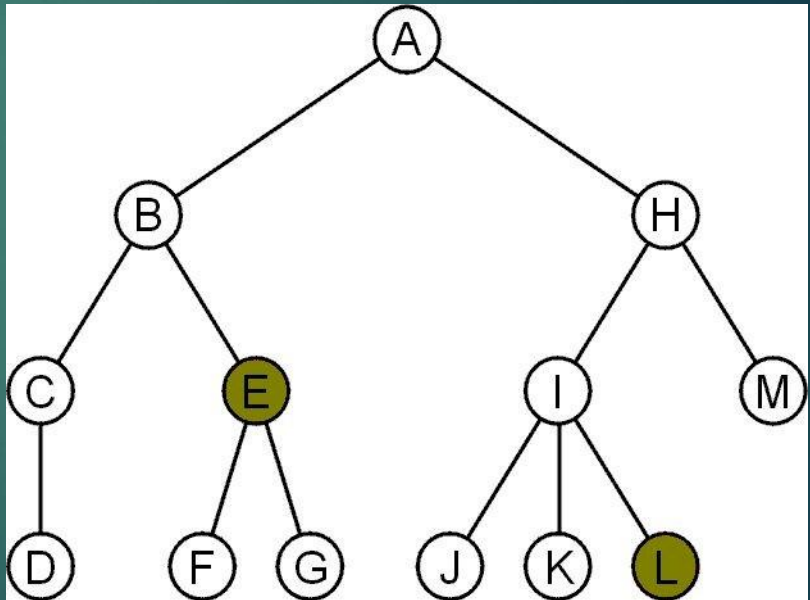


Terminology

For each node in a tree, there exists a unique path from the root node to that node

The length of this path is the *depth* of the node, e.g.,

- E has depth 2
- L has depth 3



Terminology

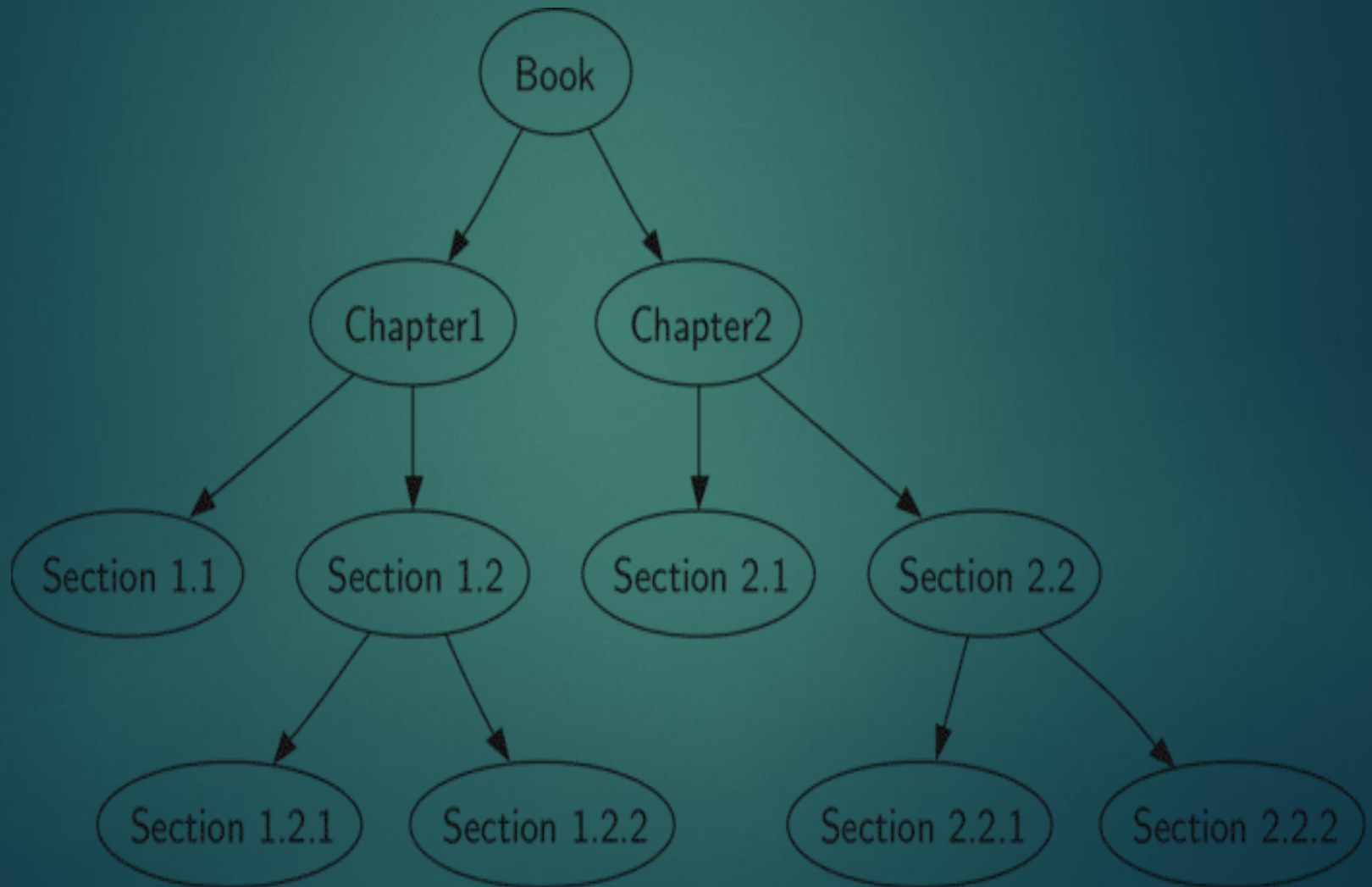
The *height* of a tree is defined as the maximum depth of any node within the tree

The height of a tree with one node is 0

- Just the root node

For convenience, we define the height of the empty tree to be -1

Terminology



Terminology

If a path exists from node a to node b :

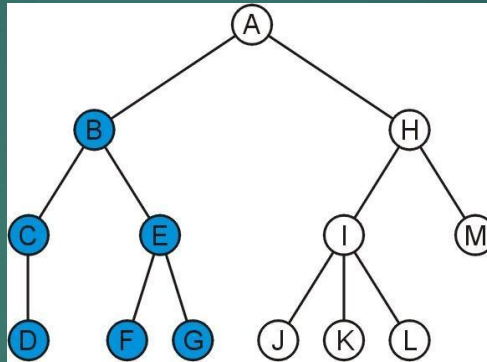
- a is an *ancestor* of b
- b is a *descendent* of a

Thus, a node is both an ancestor and a descendant of itself

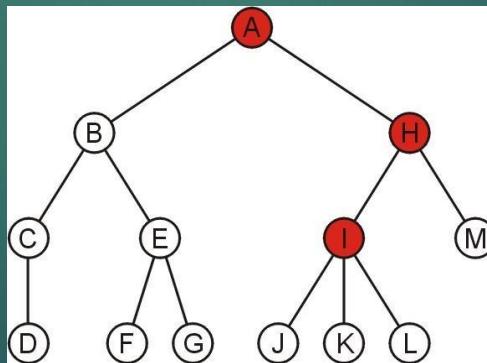
The root node is an ancestor of all nodes

Terminology

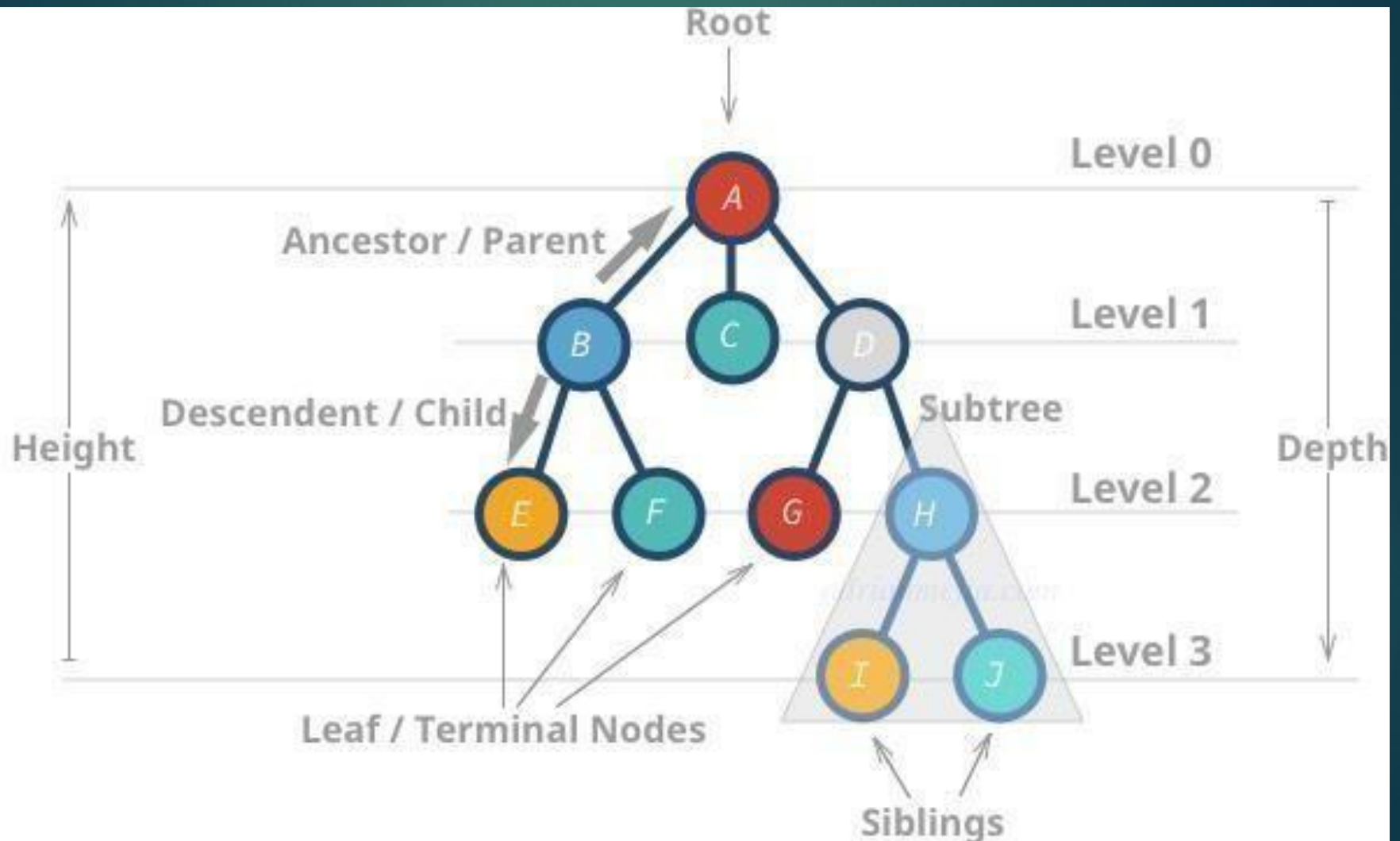
The descendants of node B are B, C, D, E, F, and G:



The ancestors of node I are I, H, and A:



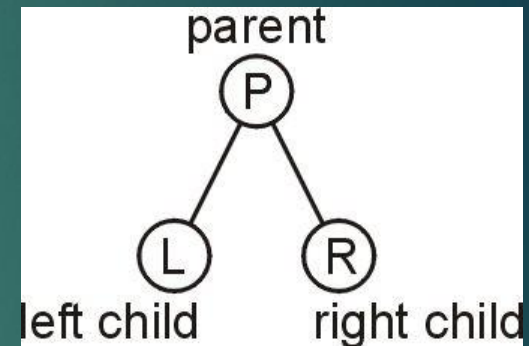
Summarized Terminologies



A Binary Tree

A binary tree is a restriction where each node has exactly two children:

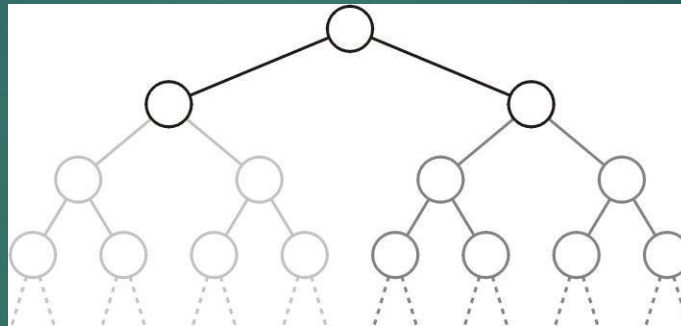
- Each child is either empty or another binary tree
- This restriction allows us to label the children as *left* and *right* subtrees



Binary Sub-trees

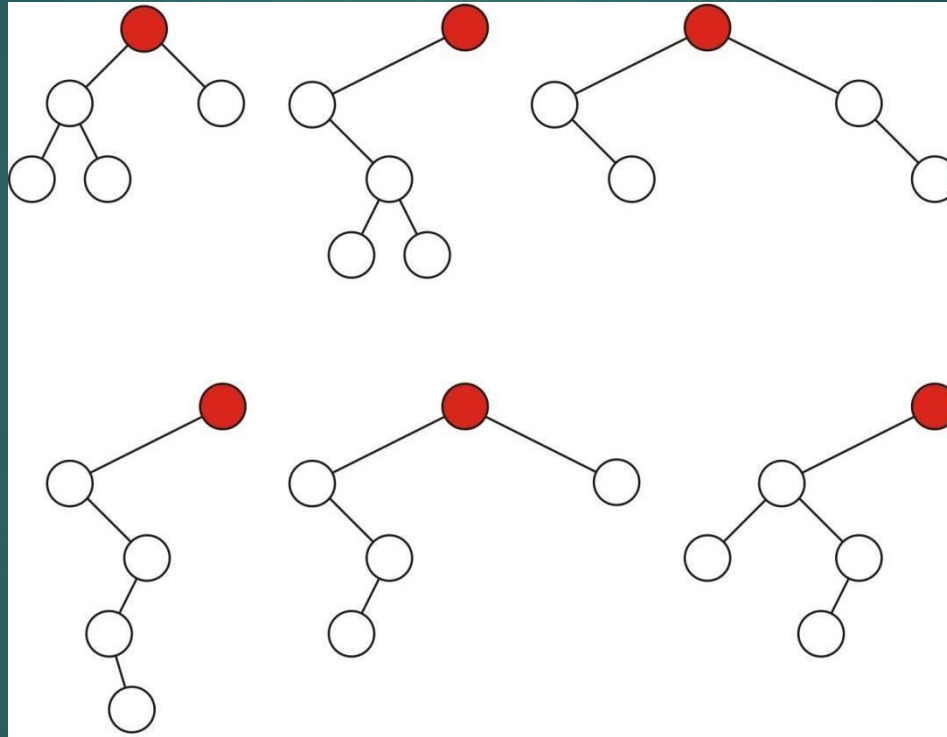
We will also refer to the two sub-trees as

- The left-hand sub-tree, and
- The right-hand sub-tree



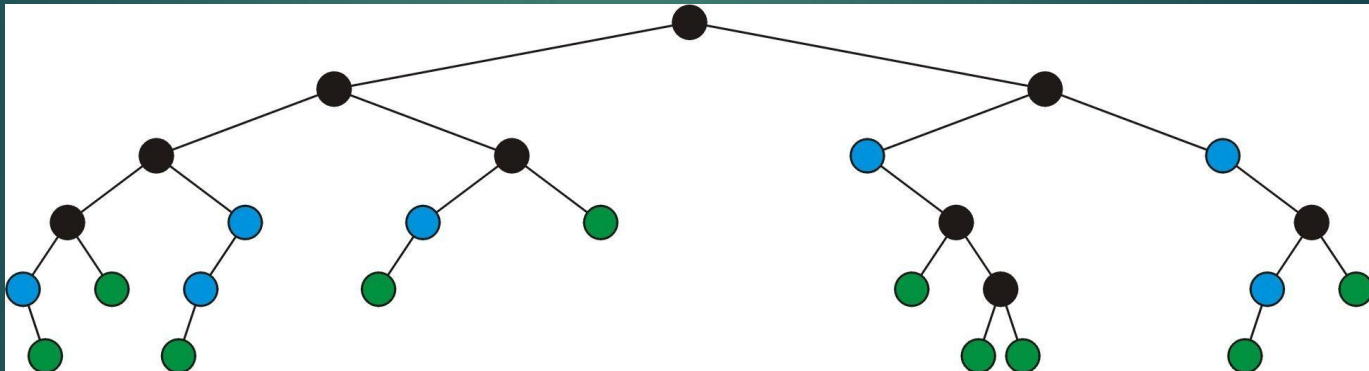
Sample Binary Trees

Sample variations on binary trees with five nodes:



Definition (Full Node)

A *full* node is a node where both the left and right sub-trees are non-empty trees



Legend:

full nodes



neither

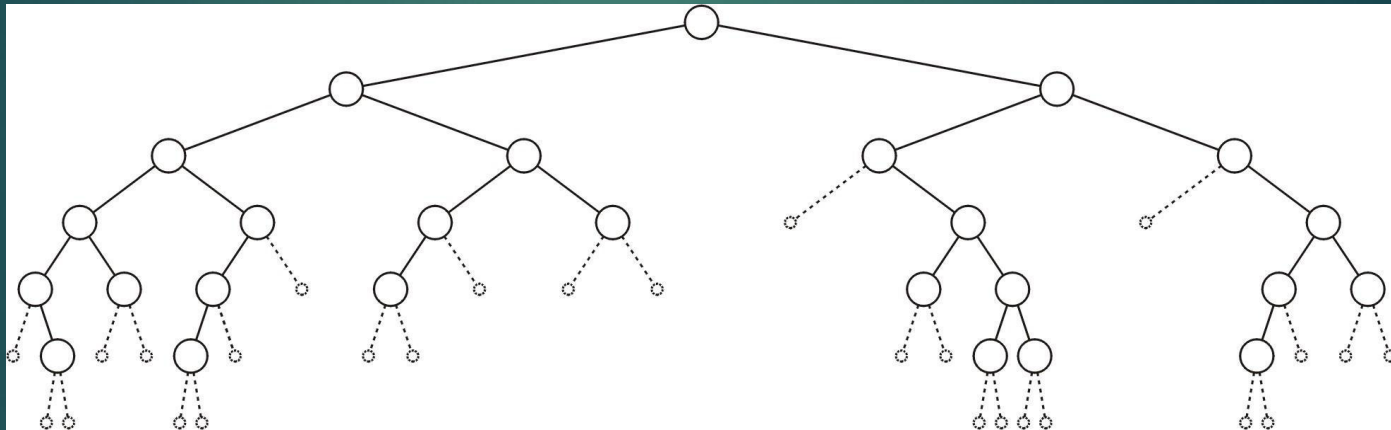


leaf nodes



Definition(Empty Node)

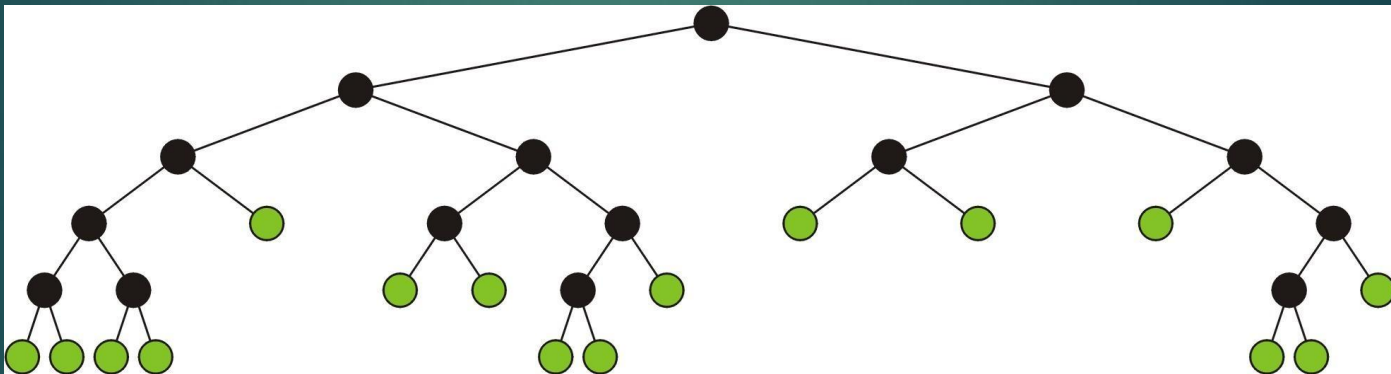
An *empty node* or a *null sub-tree* is any location where a new leaf node could be appended



Full Binary Tree

A *full binary tree* is where each node is:

- A full node, or
- A leaf node



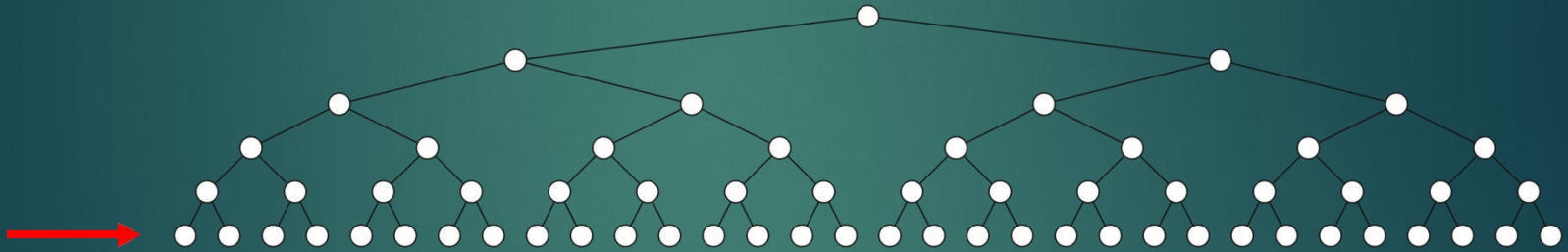
These have applications in

- Expression trees
- Huffman encoding

Perfect Binary tree

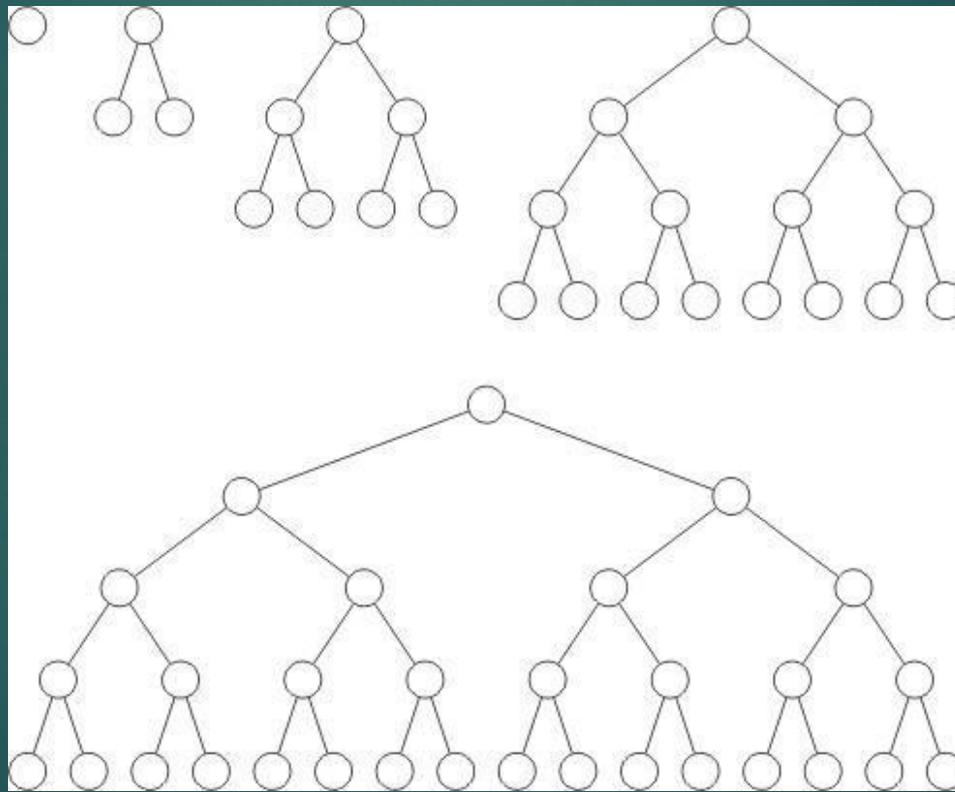
Standard definition:

- A perfect binary tree of height h is a binary tree where
 - All leaf nodes have the same depth h
 - All other nodes are full



Examples

Perfect binary trees of height $h = 0, 1, 2, 3$ and 4



Perfect Binary trees

Perfect binary trees are considered to be the *ideal* case

- The height and average depth are both $\Theta(\ln(n))$

We will attempt to find trees which are as close as possible to perfect binary trees

One of the limitations of perfect binary trees is restricted number of nodes.

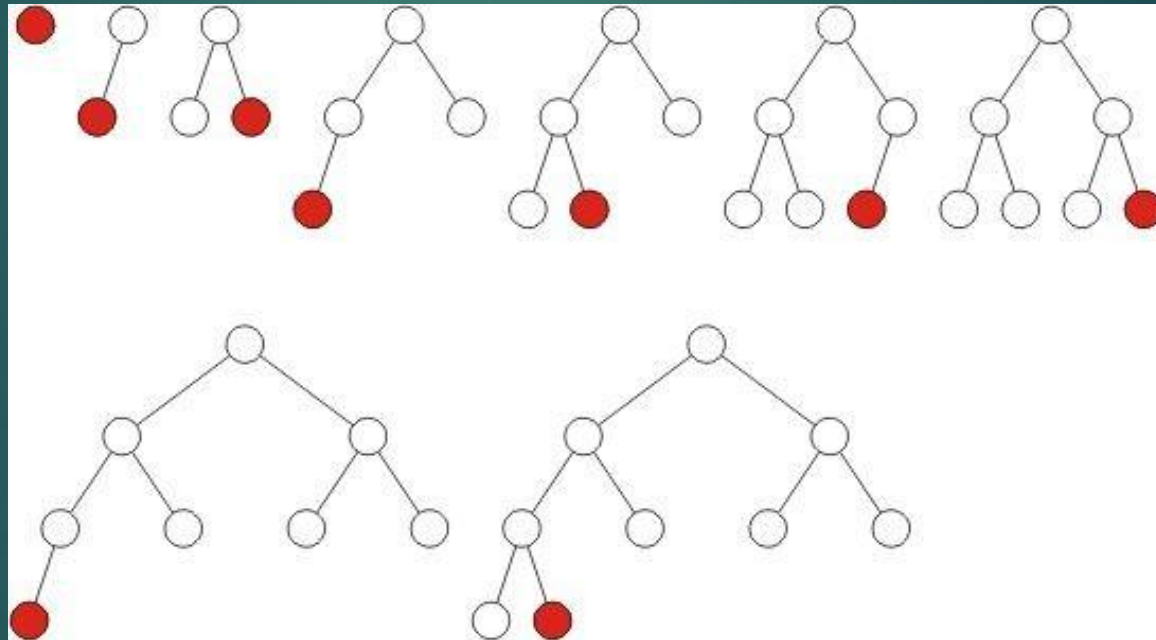
Complete Binary trees

We require binary trees which are

- Similar to perfect binary trees, but
- Defined for all n

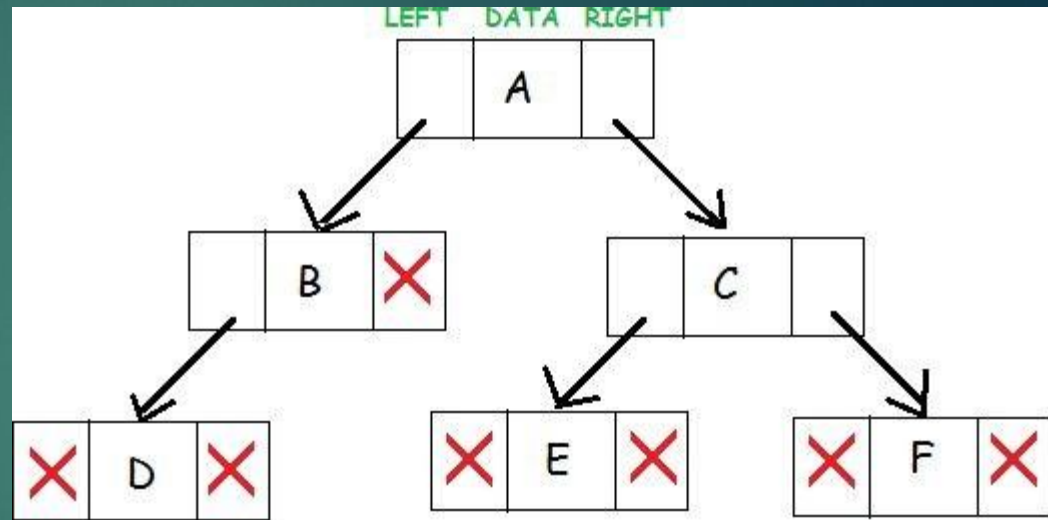
Complete Binary Trees

A complete binary tree filled at each depth from left to right:



Implementation Details Structure of a Node

```
Struct
node{ int
data;
node*
right;
node* left;
};
Struct node* newNode(int
value){
    node->data=valu
e;
    Node->left=NULL
;
    Node->right=NUL
L; Return (node);
}
```



BT Types Based on Structure

Type	Description
Full Binary Tree	Every node has 0 or 2 children (no node has only one child).
Complete Binary Tree	All levels are completely filled except possibly the last , which is filled from left to right .
Perfect Binary Tree	All internal nodes have two children, and all leaves are at the same level .
Balanced Binary Tree	The height difference between left and right subtrees of any node is not more than 1 . (e.g., AVL Tree, Red-Black Tree)
Degenerate (or Skewed) Binary Tree	Each parent has only one child , making it like a linked list . – Left-skewed or Right-skewed.

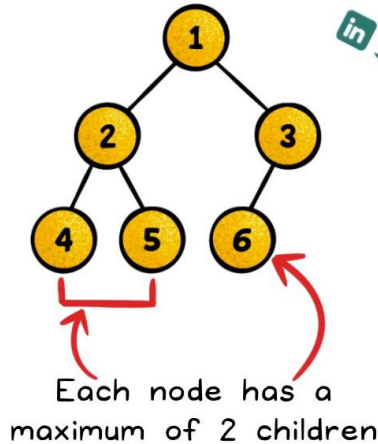
BT Types Based on Node Values (Special Properties)

Type	Description
Binary Search Tree (BST)	Left subtree < root < right subtree (ordered property).
AVL Tree	Self-balancing BST with balance factor = -1, 0, or 1.
Red-Black Tree	Self-balancing BST using color properties (Red/Black).
Splay Tree	Self-adjusting BST — recently accessed nodes are moved to the root.
Treap	Combines BST and heap properties (using key and priority).
Segment Tree	Used for range queries (sum, min, max, etc.).
Fenwick Tree (Binary Indexed Tree)	Used for prefix sums and frequency tables .
Expression Tree	Internal nodes are operators , leaves are operands .

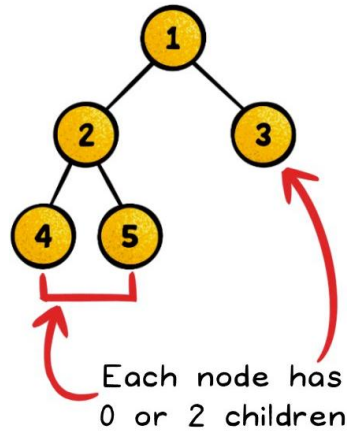
BT Types Based on Node Values (Special Properties)

Type	Description
Binary Search Tree (BST)	Left subtree < root < right subtree (ordered property).
AVL Tree	Self-balancing BST with balance factor = -1, 0, or 1.
Red-Black Tree	Self-balancing BST using color properties (Red/Black).
Splay Tree	Self-adjusting BST — recently accessed nodes are moved to the root.
Treap	Combines BST and heap properties (using key and priority).
Segment Tree	Used for range queries (sum, min, max, etc.).
Fenwick Tree (Binary Indexed Tree)	Used for prefix sums and frequency tables .
Expression Tree	Internal nodes are operators , leaves are operands .

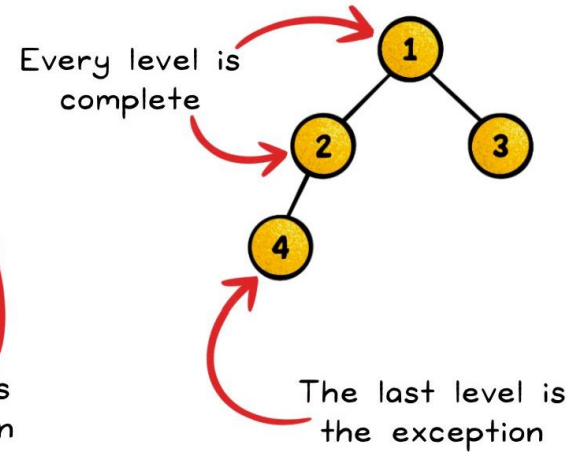
Binary tree



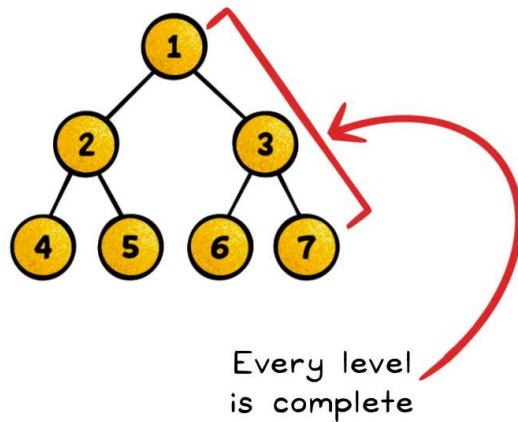
Full binary tree



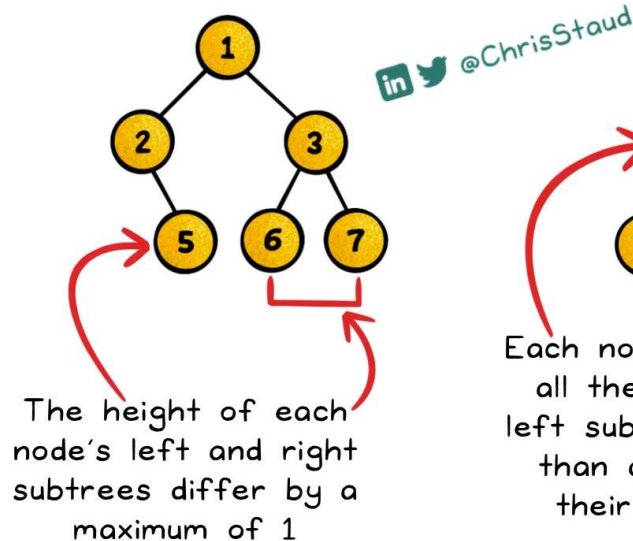
Complete binary tree



Perfect binary tree



Balanced binary tree



Binary search tree

